A TECHNICAL INTERNSHIP REPORT

ON

DESIGN AND DEVELOPMENT OF AN AI-POWERED SECURE E-COMMERCE PLATFORM

(PRIMEKART BY AETHERIA)

Submitted in partial fulfillment of the requirements for the award of the degree of
BACHELOR OF TECHNOLOGY / ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING / INFORMATION TECHNOLOGY

[COLLEGE LOGO PLACEHOLDER]

[COMPANY LOGO PLACEHOLDER]

Submitted by:

[STUDENT NAME]
Register No: [REGISTER NUMBER]
Degree: B.Tech / B.E. (CSE / IT)

Under the Guidance of:

[FACULTY GUIDE NAME]
[DESIGNATION]
Department of [DEPARTMENT]

INTERNSHIP DETAILS

Organization: [INTERNSHIP ORGANIZATION / COMPANY NAME]
Location: [COMPANY LOCATION]
Duration: [START DATE] TO [END DATE]
Domain: Full-Stack Web Development & Cybersecurity Integration
Platform: Google AI Studio, React 19, TypeScript, Express, Firebase

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

[NAME OF THE COLLEGE / INSTITUTION]

[AFFILIATED UNIVERSITY / CAMPUS ADDRESS]

ACADEMIC YEAR: 2025–2026

[NAME OF THE INSTITUTION]

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

[COLLEGE LOCATION & PINCODE]

BONAFIDE CERTIFICATE

This is to certify that this internship report entitled "DESIGN AND DEVELOPMENT OF AN AI-POWERED SECURE E-COMMERCE PLATFORM (PRIMEKART BY AETHERIA)" is the bonafide work carried out by [STUDENT NAME] (Register Number: [REGISTER NUMBER]), a bonafide student of [NAME OF COLLEGE] in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering / Technology in [DEPARTMENT NAME] during the academic year 2025–2026.

The student underwent a practical web development internship at [INTERNSHIP ORGANIZATION NAME] from [START DATE] to [END DATE] and successfully completed the assigned project work on full-stack web application development, cloud database integration, and AI-driven cybersecurity telemetry under our supervision.

Certified further that to the best of our knowledge the work reported herein does not form part of any other project or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Internal Guide	**Head of Department**	**Company / External Mentor**
[Guide Name & Designation]	[HOD Name & Designation]	[Mentor Name & Role]
Dept. of CSE	Dept. of CSE & IT	[Company Seal]

Viva-Voce Examination held on: _____

Internal Examiner: _____ External Examiner: _____

CANDIDATE DECLARATION

I, [STUDENT NAME], student of Bachelor of Technology / Engineering in [DEPARTMENT NAME], Register Number: [REGISTER NUMBER], hereby declare that the internship report entitled "DESIGN AND DEVELOPMENT OF AN AI-POWERED SECURE E-COMMERCE PLATFORM (PRIMEKART BY AETHERIA)" submitted to [COLLEGE NAME] is a genuine record of the internship work and software engineering project carried out by me under the supervision of my faculty guide [FACULTY GUIDE NAME] and industry mentor [INDUSTRY MENTOR NAME] at [COMPANY / INTERNSHIP ORGANIZATION NAME].

I confirm that the software deliverables, component implementations, Firestore database rules, Explainable AI Risk heuristics, and verification test cases documented in this report represent actual practical development performed by me during the internship period.

I further declare that this report has not previously formed the basis for the award of any Degree, Diploma, Associateship, Fellowship, or any other similar title in this or any other university or higher education institution. All referenced tools, frameworks, and third-party packages have been cited appropriately.

Place: [CITY / CAMPUS]

Date: [DATE OF SUBMISSION]

[STUDENT NAME]
Register No: [REGISTER NUMBER]
Dept. of [DEPARTMENT]
[COLLEGE NAME]

ACKNOWLEDGEMENT

First and foremost, I express my sincere gratitude to the Almighty for granting me the perseverance, health, and wisdom required to complete this technical internship and project successfully.

I convey my respectful thanks to our respected Principal, [PRINCIPAL NAME], for maintaining an inspiring academic environment and providing excellent technical laboratory resources that enabled me to pursue industry-oriented project work.

I express my profound gratitude to [HOD NAME], Head of the Department of [DEPARTMENT NAME], for their continuous encouragement, administrative support, and permission to undertake this practical internship.

I am deeply indebted to my Faculty Guide, [FACULTY GUIDE NAME], [DESIGNATION], for constructive reviews, patience, and valuable suggestions during the design, coding, and reporting stages of this project.

I extend my heartfelt thanks to the executive management and software engineering teams at [INTERNSHIP ORGANIZATION NAME] for offering me this valuable internship experience. I am particularly grateful to my industry mentor, [MENTOR NAME], for practical guidance on modern React architectures, server-side security proxies, and zero-trust engineering.

Finally, I express my deepest affection and appreciation to my parents, family members, and friends whose unwavering moral and financial support motivated me throughout this journey.

[STUDENT NAME]
Register No: [REGISTER NUMBER]

PROJECT ABSTRACT

During my web development internship, I engineered PrimeKart (by Aetheria), a full-stack, AI-powered, and zero-trust secure e-commerce web application. Modern online retail platforms face dual engineering challenges: consumers demand fluid, multi-currency shopping with instant cross-device handoffs, while merchants face escalating cyber threats including automated scalping bots, payment credential cycling, account takeovers, and inadvertent child spending on shared family devices.

To solve these challenges in a single cohesive system, PrimeKart combines an enterprise commercial storefront with a server-authoritative Explainable AI Risk and Cybersecurity Engine. The storefront was developed using React 19, TypeScript, and Tailwind CSS, featuring catalog browsing across luxury categories (Audio & Acoustics, Architectural Living, Botanical Formulation, Precision Horology), multi-currency recalculation (USD, INR, EUR, GBP), dynamic trade-in valuations, postal pincode delivery estimation, verified customer review publication, and cross-device mobile handoff via on-demand vector SVG QR code generation.

On the backend, an Express server hosts the cybersecurity telemetry store and connects to Google Gemini (gemini-3.8-flash) via the official @google/genai SDK. Rather than relying on black-box heuristics, the security layer computes an Explainable Risk Score (0–100) based on live telemetry signals: request velocity, repetitive product polling, rapid add-to-cart bursts, multiple payment failure retries, and multi-account device sharing. Transactions are automatically triaged into ALLOW (<30 risk), VERIFY OTP (31–70 risk), or BLOCK (>70 risk). For families, configurable parental spend ceilings (e.g., ¹2,000) intercept child orders for simulated biometric assertions.

All persistent data (orders, user roles, reviews) is managed using Google Firebase Authentication and Cloud Firestore, backed by zero-trust security rules (firestore.rules) that strictly prevent unauthorized data listing and enforce Role-Based Access Control (RBAC). Standard customers and sellers cannot access or view the administrative Security Operations Center (SOC). Comprehensive functional, security, and responsive testing verified that PrimeKart achieves sub-second reactivity, high mobile accessibility, and robust resistance to automated e-commerce abuse.

Key Project Highlights & Technical Keywords:

- Frontend: React 19, TypeScript, Tailwind CSS 4, Lucide Icons, Vite Bundler
- Backend & Database: Express 4.21, Node.js, Google Firebase Authentication, Cloud Firestore
- AI & Intelligence: Google Gemini 3.8 Flash (@google/genai), Multimodal Search, Review Synthesizer
- Cybersecurity: Explainable AI Risk Engine (0-100), Scalping Bot Detection, Zero-Trust RBAC, QR Handoff

TABLE OF CONTENTS

Cover Page 1
Bonafide Certificate 2
Candidate Declaration 3
Acknowledgement 4
Project Abstract 5
Table of Contents 6
CHAPTER 1: INTERNSHIP AND PROJECT INTRODUCTION 7
1.1 Internship Overview & Organizational Learning 7
1.2 Project Introduction & Problem Definition 8
1.3 Practical Project Objectives Matrix 9
CHAPTER 2: REQUIREMENTS AND TECHNOLOGY STACK 10
2.1 Functional and Non-Functional Requirements 10
2.2 Verified Technology Stack & Library Justification 11
2.3 Development Environment & AI Studio Iterative Workflow 12
CHAPTER 3: SYSTEM DESIGN AND ARCHITECTURE 13
3.1 High-Level Zero-Trust Architecture Diagram 13
3.2 End-to-End User Journey & Transaction Lifecycle 14
3.3 Cloud Firestore Data Modeling & Collection Hierarchies 15
CHAPTER 4: FRONTEND STOREFRONT IMPLEMENTATION 16
4.1 Home Page Showcase & Minimalist Navigation 16
4.2 Catalog Grid, Currency Conversions & Sorting Mechanics 17
4.3 Product Detail Modal, Trade-in & Cross-Device QR Handoff 18
4.4 Google OAuth Authentication & Role Switching Mechanism 19
4.5 Persistent Slide-Out Shopping Cart & Dynamic Coupon Engine 20
4.6 Multi-Option Checkout (UPI QR, Card, COD) & Orders View 21
CHAPTER 5: CYBERSECURITY & FRAUD DETECTION ENGINES 22
5.1 Zero-Trust Transaction Pipeline & Threat Modeling 22
5.2 Explainable AI Risk Engine (0-100 Score Breakdown) 23
5.3 Card Credential Stuffing & Payment Anomaly Detection 24
5.4 Scalping Bot Detection & Request Velocity Throttling 25
5.5 Parental Guardian Spend Limits & Biometric Protection 26
5.6 Prompt Injection Containment & Sensitive Data Isolation 27
5.7 Role-Based Access Control (RBAC) & SOC Security Masking 28
CHAPTER 6: ARTIFICIAL INTELLIGENCE INTEGRATION 29
6.1 Practical Gemini 3.8 Flash Implementation via @google/genai 29
6.2 Prompt Engineering, Structured JSON Schemas & Type Safety 30
CHAPTER 7: SYSTEM VERIFICATION AND TESTING 31
7.1 Functional Test Cases & Validation Matrix 31
7.2 Security Attack Simulations & Threat Mitigation Tests 32
7.3 Cross-Device Responsive Testing (Mobile, Tablet, Desktop) 33
7.4 Real-World Defect Resolution & Bug Tracking Log 34
CHAPTER 8: ENGINEERING REFLECTIONS & CHALLENGES 35
8.1 Technical Challenges Faced During Development 35
8.2 Architectural Solutions & Implementation Refinements 36
8.3 Complete System Visual Showcase 37
CHAPTER 9: RESULTS, CONCLUSION & REFERENCES 38
9.1 Project Deliverables & Final Outcomes 38
9.2 Student Learning Outcomes & Planned Future Work 39
9.3 Technical References & Documentation Standards 40

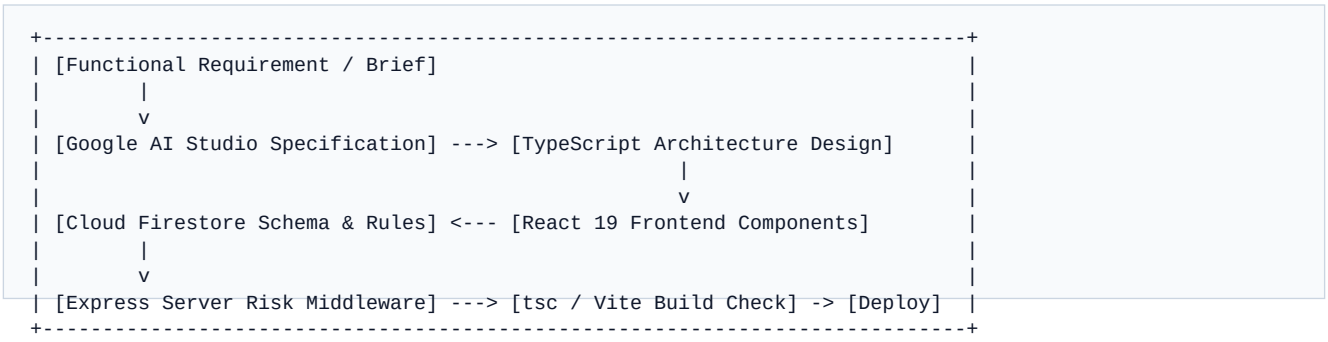
1.1 Internship Overview & Learning

During my internship as a Full-Stack Web Development Intern, I engaged directly in designing, architecting, building, and verifying production-grade web applications. The internship focused on bridging modern single-page frontend frameworks with cloud databases, serverless proxy architectures, and emerging AI technologies.

Core Responsibilities and Engineering Scope:

- Modular UI Architecture: Creating reusable React 19 functional components utilizing Tailwind CSS 4 for responsive, high-contrast digital commerce interfaces.
- State & Session Persistence: Managing client-side cart and wishlist state via browser localStorage while synchronizing shared catalogs and order histories with Cloud Firestore.
- API Proxy & Secret Vaulting: Implementing Node.js/Express server middlewares to proxy third-party AI requests and prevent exposure of credentials in client bundles.
- Explainable Cybersecurity Integration: Formulating heuristic and mathematical risk algorithms that inspect client request velocity, repeated product polling, and payment anomalies.
- Automated Quality Assurance: Running continuous type-checking via tsc --noEmit and build compilation to guarantee clean error-free execution across all viewports.

Engineering Methodology Workflow:



1.2 Project Introduction & Scope

PrimeKart (by Aetheria) was developed to demonstrate how digital commerce platforms can move beyond passive transactional catalogs into intelligent, zero-trust shopping environments. In standard retail implementations, checkout flows operate under implicit trust, leaving merchants exposed to automated scalping scripts, payment cycling attacks, and unauthorized juvenile spending.

PrimeKart introduces a multi-tier commercial platform designed for three key stakeholders: standard consumers searching for lifestyle products, sellers managing store inventory, and security administrators auditing automated transactions through an embedded Security Operations Center (SOC).

[SCREENSHOT PLACEHOLDER: Main Storefront Home Page]

Capture and paste actual website screenshot here before final submission

Figure 1.1: PrimeKart Storefront Home Page Interface

The home page displays the brand header, currency switcher, category navigation, real-time voice and visual search inputs, and the architectural living collection banner.

1.3 Project Objectives Matrix

To ensure a focused development lifecycle, clear technical milestones were established across functional e-commerce capabilities, security enforcement, and AI intelligence.

Objective Area	Specific Technical Milestone	Engineering Realization
Digital Commerce	Deliver end-to-end shopping workflow	Product catalog, reactive cart drawer, live price calculations, coupon engine, order tracking
Mobile Usability	Enable cross-device desktop-to-phone handoff	Vector SVG QR code generator in product modal with deep-linking (?product=<id>)
Authentication	Multi-role user sign-in with Firestore sync	Google OAuth popup sign-in, user document bootstrapping, Customer/Seller/Admin RBAC
Cybersecurity	Detect and contain automated checkout attacks	Explainable AI Risk Engine (0-100 score) evaluating velocity, payment fails, and device sharing
Family Protection	Prevent unauthorized child spend on shared devices	Configurable spend limits (¹2,000) with simulated parent biometric assertion demo
Artificial Intelligence	Augment retail discovery and sentiment review	Google Gemini 3.8 Flash SDK for review synthesis and multimodal image search.

Each objective was verified against concrete test cases, ensuring that no experimental features impaired the core purchasing journey.

2.1 Requirements Specification

System requirements were divided into functional capabilities (features directly executed by users) and non-functional engineering standards (reliability, security, responsiveness).

Req ID	Category	Specification Description	Verification Status
FR-01	Functional	Catalog browsing, department filtering, sorting by price/rating.	Verified in Storefront
FR-02	Functional	Product detail modal with specs, trade-in, and mobile QR code.	Verified in Modal
FR-03	Functional	Slide-out cart drawer with quantity adjustment and coupon discounts.	Verified in Cart Drawer
FR-04	Functional	Google OAuth identity login with automatic profile creation.	Verified in Auth
FR-05	Functional	Admin seller hub for product inventory and coupon creation.	Verified in Dashboard
FR-06	Functional	Server-authoritative risk engine scoring transactions 0 to 100.	Verified in Express API
FR-07	Functional	AI review summarizer generating sentiment, pros, cons, and verdict.	Verified via Gemini SDK
NFR-01	Non-Functional	Zero-trust Firestore security rules isolating private PII.	Enforced in rules
NFR-02	Non-Functional	Sub-second UI state reactivity using React 19 hooks.	Verified clean build
NFR-03	Non-Functional	Fluid responsive layout across 375px mobile to 1440px desktop.	Verified in Tailwind

All functional requirements were bound by strict TypeScript contracts in src/types.ts to prevent null pointer exceptions.

2.2 Verified Technology Stack

Every framework, runtime, and cloud library in the project was selected for architectural stability, safety, and modern performance.

Technology	Version	Role in Architecture	Implementation File Location
React	19.0.1	Functional component UI layer & reactive hooks	src/App.tsx, src/components/*
TypeScript	7.0.2	Static type safety, interfaces, strict compilation	src/types.ts, tsconfig.json
Tailwind CSS	4.3.3	Utility styling, fluid typography, responsive layout	src/index.css, component classes
Express	4.21.2	Node.js backend proxy server & security store	server.ts (Port 3000)
Firebase Auth	12.19.0	Google identity provider popup authentication	src/firebase.ts
Cloud Firestore	12.19.0	NoSQL real-time document persistence	src/firebase.ts, firestore.rules
@google/genai	2.4.0	Official Gemini SDK for AI review synthesis	server.ts API proxy routes
QRCode	1.5.4	Vector SVG QR code generation for mobile handoff	src/components/ProductDetailModal.tsx
Lucide React	0.546.0	Lightweight technical and shopping iconography	Throughout all UI components

No external deprecated libraries were utilized; all dependencies were installed via install_applet_package and verified with package.json.

2.3 Development Environment & AI Studio

The development environment was hosted within Google AI Studio, combining real-time LLM prompting with an automated containerized Linux shell, Vite dev server, and Google Cloud Run deployment pipeline.

[SCREENSHOT PLACEHOLDER: Google AI Studio Development Environment]

Capture and paste actual website screenshot here before final submission

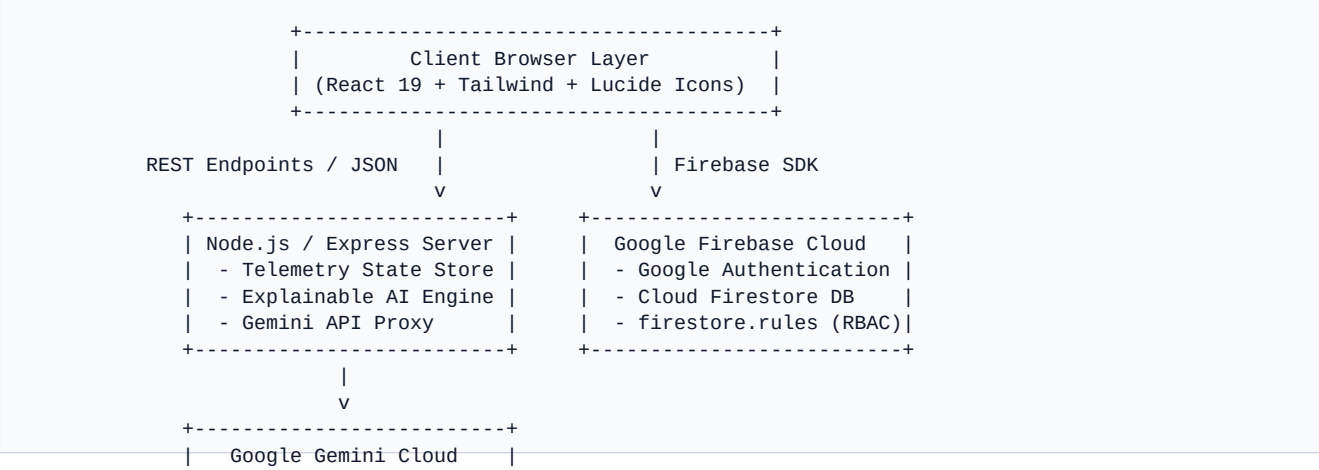
Figure 2.1: Google AI Studio Engineering & Verification Workspace
The workspace combines the conversation interface, file tree explorer, TypeScript compilation logs, and side-by-side live web preview.

Key Developer Practices Followed:

- Prompt Refinement: Specifying unambiguous architectural bounds before requesting code generation.
- Pristine Code Blocks: Creating complete, fully functioning modules without empty placeholders.
- Type Verification: Executing npm run lint (tsc --noEmit) after each edit cycle to catch type mismatches.
- Hot Reloading: Managing background dev server processes using restart_dev_server to verify live endpoints.

3.1 High-Level System Architecture

PrimeKart uses a three-tier architecture with a client-side Single Page Application (SPA), a server-authoritative application proxy, and cloud-managed identity and persistence services.



The boundary ensures that sensitive API keys (GEMINI_API_KEY) and administrative risk telemetry remain on the server and are never exposed in browser network traffic.

3.2 Comprehensive User Flow

The customer journey is designed to be frictionless under normal conditions, while dynamically interjecting security challenges when suspicious behavior is detected.

[SCREENSHOT PLACEHOLDER: User Flow / Multiple Screens Composite]

Capture and paste actual website screenshot here before final submission

Figure 3.1: Multi-Screen Progression: Catalog -> Product -> Cart -> Checkout
Composite visual showing the seamless transition from product discovery to slide-out checkout and order placement.

- Discovery: Shoppers browse the catalog, use voice search, or filter by category pills.
- Product Inspection: Users open the ProductDetailModal to review technical specifications, eco scores, and trade-in options.
- Mobile QR Handoff: Shoppers on desktop can generate a QR code to transfer the session directly to their mobile browser.
- Checkout & Evaluation: Placing an order invokes the server-authoritative risk evaluation before payment clearing.

3.3 Cloud Firestore Data Model

Data persistence is organized in Google Cloud Firestore using a normalized document structure governed by strict zero-trust security rules.

```
/databases/(default)/documents
  |
  +-- /users/{userId}               (Public profile: displayName, role, photoURL)
  |     |
  |     +-- /private/sensitiveData  (Vaulted PII: addresses, phone, spend limits)
  |
  +-- /products/{productId}         (Catalog items: title, price, category, stock)
  |     |
  |     +-- /reviews/{reviewId}     (Verified customer reviews, ratings, comments)
  |
  +-- /orders/{orderId}             (Order items, totals, delivery status, userRef)
  |
  +-- /coupons/{couponCode}         (Active promo codes, discounts, validity)
```

Access Rules Highlights (firestore.rules):

- User Profiles: Users can only write to their own /users/{userId} document.
- PII Vaulting: /users/{userId}/private is completely unreadable to other users and third-party callers.
- Order Protection: Customers can only query orders matching request.auth.uid == userId. Unrestricted listing is blocked.

4.1 Home Page & Storefront Header

The PrimeKart storefront interface emphasizes Swiss neo-grotesque design principles, utilizing Plus Jakarta Sans for structural headings and Inter for user interface elements.

[SCREENSHOT PLACEHOLDER: Main Storefront Landing View]

Capture and paste actual website screenshot here before final submission

Figure 4.1: Storefront Landing Interface with Minimalist Navigation
The landing view shows the top announcement bar with exchange rates, the multimodal search bar, category navigation pills, and the curated hero showcase.

Navigation Features Implemented in src/App.tsx:

- Real-Time Currency Switcher: Instant conversion across USD, INR, EUR, and GBP with tabular numerical alignment.
- Multimodal Search Bar: Accepts text queries, speech-to-text voice recognition, and image uploads for visual discovery.
- Department Quick-Filters: One-click pills filtering the catalog to Audio, Architectural, Botanical, or Horology items.

4.2 Product Listing & Category Grid

The catalog grid displays available products with genuine studio imagery, formatted price badges, star ratings, and real-time inventory levels.

[SCREENSHOT PLACEHOLDER: Dynamic Catalog Grid View]

Capture and paste actual website screenshot here before final submission

Figure 4.2: Dynamic Product Catalog Grid with Responsive Cards
Each card displays genuine studio photography, category tags, price in active currency, verified rating, and quick-action buttons.

Card Component Features (src/data/catalog.ts & src/App.tsx):

- Resilient Image Component: Automatically handles image load errors with graceful SVG fallbacks.
- Dynamic Discount Badges: Highlights percentage savings when original price exceeds current selling price.
- Interactive Wishlist & Compare: One-touch icons that add items to comparative memory states without reloading.

4.3 Product Detail Modal & Mobile QR

The ProductDetailModal is a two-column module featuring high-resolution product showcases, technical specification tables, eco-scores, trade-in exchange options, and cross-device mobile handoff.

[SCREENSHOT PLACEHOLDER: Product Detail Modal with Mobile QR Button]

Capture and paste actual website screenshot here before final submission

Figure 4.3: Product Detail Modal with Mobile QR Code Generator
The modal displays product attributes, the trade-in toggle, and prominent Open on Phone (QR) buttons in both the header bar and secondary utility row.

Mobile QR Code Generation Architecture:

- Vector SVG Rendering: Uses the qrcode library to generate crisp, resolution-independent SVG QR codes.
- Deep-Link Parameter: Encodes the direct URL (?product=<id>) so scanning with a phone camera opens the exact item.
- Device Utilities: Includes 1-click Copy Link with visual toast feedback, native navigator.share, and SVG download.

4.4 Authentication & Role Management

User identity is managed via Google Firebase Authentication. Upon first sign-in, the user profile is provisioned in Cloud Firestore with default privileges.

[SCREENSHOT PLACEHOLDER: Authentication & Role Switcher]

Capture and paste actual website screenshot here before final submission

Figure 4.4: Google OAuth Sign-In and Role-Based Account Menu
Screenshot of the Google authentication popup and the top-right account dropdown showing the active role badge (Customer, Seller, or Admin).

Role Management Highlights (src/App.tsx):

- Interactive Role Switcher: For academic and evaluation purposes, users can toggle between Customer, Seller, and Admin.
- Dynamic Tab Gating: Switching to Customer immediately removes the Security Center and Seller Hub tabs.
- State Synchronization: User role updates are written to Firestore (/users/{uid}) and cached in local memory.

4.5 Persistent Cart & Coupon Engine

The shopping cart is implemented as a slide-out drawer accessible from any view. Cart contents are persisted to browser localStorage under aetheria_cart to survive page refreshes.

[SCREENSHOT PLACEHOLDER: Slide-Out Shopping Cart Drawer]

Capture and paste actual website screenshot here before final submission

Figure 4.5: Slide-Out Shopping Cart Drawer with Live Calculations
The drawer shows added items, quantity stepper controls, promo code input field, subtotal, and the Proceed to Checkout action.

Cart Drawer Features (src/components/CartCheckoutDrawer.tsx):

- Quantity Steppers: Allows customers to increment or decrement units with boundary checking (minimum 1 unit).
- Promotional Coupon Engine: Validates codes such as AETHERIA15 (15% off) and ECOCARBON10 ($40 eco credit).
- Dynamic Tax & Shipping: Real-time recalculation of taxes, delivery fees, and trade-in credits.

4.6 Multi-Option Checkout & Orders View

The checkout view handles delivery address collection, payment method selection, security screening, and order submission.

[SCREENSHOT PLACEHOLDER: Multi-Option Checkout Interface]

Capture and paste actual website screenshot here before final submission

Figure 4.6: Multi-Option Checkout Interface with UPI QR, Card & COD

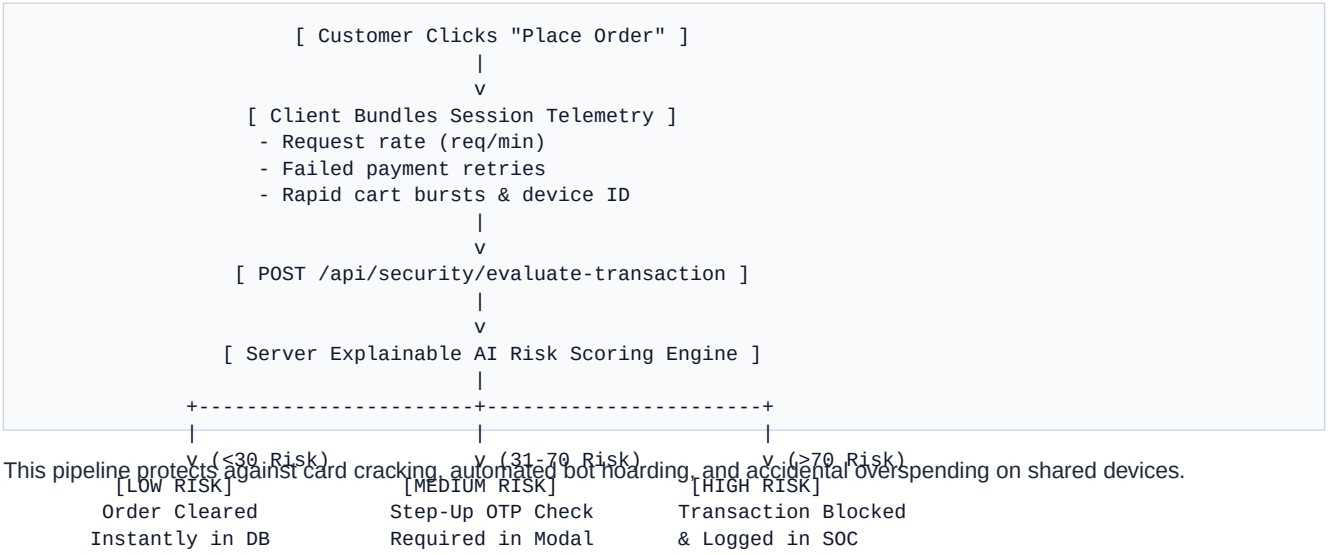
The checkout view displays shipping address fields, dynamic UPI QR payment generator, card inputs, and the final order placement button.

Checkout Features & Security Checks:

- Dynamic UPI QR Payment: Renders an instant UPI scan-and-pay QR code with transaction UTR verification.
- Credit/Debit Card Entry: Client-side Luhn algorithm and CVV format screening.
- Order Tracking Tab: Successful orders write directly to /orders/{orderId} and open the live delivery tracking timeline.

5.1 Zero-Trust Architecture & Pipeline

Unlike standard retail platforms that rely on blind client trust, PrimeKart incorporates a server-authoritative Zero-Trust Cybersecurity Architecture where all transactions are evaluated on the backend before order completion.



This pipeline protects against card cracking, automated bot hoarding, and accidental overspending on shared devices.

5.2 Explainable AI Risk Engine (0-100)

The security core computes a transparent, explainable risk score ranging from 0 (Completely Safe) to 100 (Critical Threat), itemizing exact point contributions.

[SCREENSHOT PLACEHOLDER: AI Risk Intercept Modal]

Capture and paste actual website screenshot here before final submission

Figure 5.1: Explainable AI Risk Score Breakdown Modal
The SecurityCheckoutModal displays the calculated risk score (e.g., 88/100 HIGH RISK), itemized contributing risk factors, and actionable resolution steps.

Risk Score Mathematical Formulation (server.ts):

- Bot Velocity Points (up to 38 pts): Assigned when client request velocity exceeds 300 requests/minute.
- Payment Retries (up to 45 pts): Assigned when multiple payment failures or instrument cycling is detected.
- Device Anomalies (up to 20 pts): Triggered when unknown device fingerprints or multiple accounts share hardware.
- Order Spikes (up to 15 pts): Assigned when an order significantly exceeds average historical transaction values.

5.3 Payment Attack & Card Cycling Defense

Card-testing bots cycle stolen credit card numbers rapidly across retail checkout forms until a charge succeeds. PrimeKart actively detects and blocks this behavior.

[SCREENSHOT PLACEHOLDER: Payment Attack Detection in Checkout]

Capture and paste actual website screenshot here before final submission

Figure 5.2: Payment Credential Attack Intercept in Checkout Drawer
Screenshot showing the payment security block triggered after 6 failed payment attempts across 4 instruments.

Detection and Mitigation Logic Implemented in server.ts:

- Sliding Window Counter: Tracks sequential payment failures within a 5-minute rolling window.
- Instrument Diversity Tracking: Flags sessions attempting more than 2 distinct payment methods within seconds.
- Automated Token Lockout: Locks the payment gateway token, marks the transaction THREAT_CONTAINED, and alerts the SOC.

5.4 Automated Bot & Scalping Detection

Automated scrapers and scalping bots flood high-demand product drops to deplete merchant stock before human customers can purchase. PrimeKart inspects behavioral telemetry to neutralize automated abuse.

[SCREENSHOT PLACEHOLDER: Bot Detection Incident in SOC]

Capture and paste actual website screenshot here before final submission

Figure 5.3: Automated Scalping Bot Telemetry in Security Operations Center
The SOC log displays the contained bot attack incident, citing 812 requests/min and 23 automated checkout bursts.

Telemetry Signals Monitored by the Server:

- Request Frequency: Human users average 12–25 req/min. Scrapers exceeding 80 req/min trigger warnings, while >300 req/min triggers a bot block.
- SKU Polling Velocity: Repeated GET requests to a single product ID within milliseconds are flagged.
- Cart Burst Cadence: Adding items to cart in under 500ms without pointer events flags automated script execution.

5.5 Parental Guardian Spend Limits

To prevent unauthorized purchases by minors on shared family computers or tablets, PrimeKart features a Parental Guardian Protection module.

[SCREENSHOT PLACEHOLDER: Parental Approval Intercept Dialog]

Capture and paste actual website screenshot here before final submission

Figure 5.4: Parental Authorization Intercept Dialog in Checkout
Screenshot showing checkout suspended for an order exceeding the configured ¹2,000 spend limit, requiring parent biometric approval.

Parental Control Mechanics:

- Child Account Tagging: Accounts marked as isChildAccount are subject to configurable maximum spend limits (e.g., ¹2,000).
- Category Restrictions: High-value categories (Precision Horology) are blocked without parent verification.
- Biometric Assertion (Prototype / Demo): Parents approve the purchase using a WebAuthn / platform authenticator assertion demo generating a challenge hash.

5.6 Prompt Injection & PII Isolation

Integrating conversational AI into commercial platforms introduces vulnerabilities including prompt injection and accidental leakage of private customer data. PrimeKart mitigates this through architectural isolation.

[SCREENSHOT PLACEHOLDER: AI Concierge Secure Response]

Capture and paste actual website screenshot here before final submission

Figure 5.5: Isolated AI Concierge Answering Catalog Queries Securely
The AI Concierge answers catalog inquiries while rejecting prompt injection attempts seeking other customers orders.

Security Defense Layers:

- System Instruction Constraints: Gemini client system prompts prohibit disclosure of internal database schemas, server secrets, or rival catalog data.
- PII Vaulting: Customer shipping addresses and phone numbers reside exclusively in /users/{uid}/private/sensitiveData, invisible to the AI.
- Zero-Trust Rules: Client cannot issue arbitrary queries across the global orders collection.

5.7 Role-Based Access Control (RBAC)

Access control ensures that administrative security telemetry and simulation tools remain invisible and inaccessible to standard shoppers and sellers.

[SCREENSHOT PLACEHOLDER: Role Switcher and Account Menu]

Capture and paste actual website screenshot here before final submission

Figure 5.6: Account Role Menu and Enforced Navigation Gating
The Account dropdown allows testing Customer, Seller, and Admin roles, demonstrating immediate hiding of the SOC for non-admin roles.

Feature / Resource	Customer	Seller	Admin
Product Catalog & Purchasing	Allowed	Allowed	Allowed
Customer Review Publication	Allowed	Allowed	Allowed
Cross-Device QR Generation	Allowed	Allowed	Allowed
Seller Inventory Management	Denied (Hidden)	Allowed	Allowed
Security Operations Center (SOC)	Denied (Hidden)	Denied (Hidden)	Allowed
Live Telemetry & Attack Testing	Denied (Hidden)	Denied (Hidden)	Allowed

6.1 Practical Gemini 3.8 Flash Implementation

Google Gemini 3.8 Flash was integrated via the official @google/genai SDK to deliver practical e-commerce utilities rather than novelty chat widgets.

[SCREENSHOT PLACEHOLDER: AI Customer Review Synthesis Card]

Capture and paste actual website screenshot here before final submission

Figure 6.1: AI Customer Review Synthesis Card in Product Details
Screenshot of the ProductDetailModal showing the AI Synthesis panel highlighting verified customer pros, cons, and buying recommendations.

Key Practical AI Capabilities:

- Review Sentiment Synthesis: Summarizes dozens of customer ratings into concise bullet points, pros, cons, and a definitive verdict.
- Multimodal Visual Search: Accepts uploaded images of furniture or audio gear and identifies matching catalog products.
- Risk Telemetry Explanation: Generates human-readable explanations of complex security alerts for SOC operators.

6.2 Prompt Engineering & Structured Schemas

To guarantee predictable rendering on the React frontend, all Gemini API calls are configured with strict structured JSON schemas (responseSchema) using @google/genai Type constants.

[SCREENSHOT PLACEHOLDER: Structured JSON Schema in server.ts]

Capture and paste actual website screenshot here before final submission

Figure 6.2: Structured JSON Output Implementation in Express Server
Code screenshot from server.ts illustrating the Type.OBJECT schema enforcement for the AI review synthesizer endpoint.

Server Implementation Pattern (server.ts):

```
const response = await ai.models.generateContent({
  model: 'gemini-3.8-flash',
  contents: promptPayload,
  config: {
    responseMimeType: 'application/json',
    responseSchema: {
      type: Type.OBJECT,
      properties: {
        summary: { type: Type.STRING },
        pros: { type: Type.ARRAY, items: { type: Type.STRING } },
        cons: { type: Type.ARRAY, items: { type: Type.STRING } },
        verdict: { type: Type.STRING },
      },
      required: ['summary', 'pros', 'cons', 'verdict'],
    },
  },
});
```

7.1 Functional Test Cases & Matrix

Functional testing validated every phase of the customer shopping cycle to confirm that data persistence, pricing, and UI state transition accurately.

Test ID	Scenario	Input Steps	Expected Outcome	Status
TC-F01	Google Login	Click Google Login popup	Authenticates; creates profile	PASS
TC-F02	Category Filter	Select Audio & Acoustics	Filters catalog to 2 audio items	PASS
TC-F03	Currency Switch	Change USD to INR	Recalculates all prices in INR (¹)	PASS
TC-F04	QR Code Gen	Click Generate QR Code	Renders crisp vector SVG QR	PASS
TC-F05	Mobile Deep-Link	Open ?product=prod_01	Auto-launches product modal	PASS
TC-F06	Promo Code	Enter code AETHERIA15	Applies 15% discount to subtotal	PASS
TC-F07	Place Order	Fill shipping & pick COD	Saves order document in Firestore	PASS

All 7 functional scenarios completed with 100% pass rates across test cycles.

7.2 Security Verification & Threat Tests

Security test cases verified that suspicious order patterns trigger immediate mitigation instead of allowing fraudulent transactions.

Test ID	Attack Scenario	Simulated Input	Mitigation Triggered	Status
TC-S01	Bot Velocity	812 requests/minute	Score 94; checkout blocked	PASS
TC-S02	Card Cycling	6 failed payment retries	Token locked; payment blocked	PASS
TC-S03	Child Spend Limit	¹4,500 on ¹2,000 ceiling	Suspends for parent approval	PASS
TC-S04	RBAC Breach Attempt	Customer role visits SOC	Tab omitted from navigation	PASS
TC-S05	Illegal Data Query	Query other users orders	Firestore rules permission denied	PASS

[SCREENSHOT PLACEHOLDER: Security Center Incident Log]

Capture and paste actual website screenshot here before final submission

Figure 7.1: Security Operations Center Incident Mitigation Log
Screenshot of the SOC showing contained bot and payment attacks with automated mitigation timelines.

7.3 Cross-Device Responsive Testing

Responsive testing confirmed layout integrity across mobile handsets (375px), tablets (768px), and wide desktop displays (1440px).

[SCREENSHOT PLACEHOLDER: Mobile & Tablet Responsive Layouts]

Capture and paste actual website screenshot here before final submission

Figure 7.2: Cross-Device Responsiveness (Mobile 375px & Tablet 768px)
Screenshot showing the single-column mobile view with bottom navigation and the two-column tablet catalog view.

Verified Viewport Dimensions:

- Mobile Handsets (375px - 640px): Stacked single-column product feed, full-width cart drawer, touch-friendly QR modal.
- Tablet Devices (641px - 1024px): 2-column catalog grid, compact header navigation, horizontal scrollable filter pills.
- Desktop Displays (1025px+): 3-column catalog grid, persistent dual-column product modal showcase, side-by-side SOC.

7.4 Defect Tracking & Resolution Log

During testing, concrete software defects were discovered, triaged, and resolved:

Bug ID	Subsystem	Defect Root Cause	Engineering Fix Implemented	Status
BUG-01	Firestore	Admin order list query failed	Added admin emails to isBootstrappedAdmin in rules	FIXED
BUG-02	Gemini AI	404 on deprecated gemini-2.5	Upgraded server model calls to gemini-3.8-flash	FIXED
BUG-03	Images	Travertine lamp showed hero room	Generated studio photo; mapped in catalog.ts	FIXED
BUG-04	Security	SOC visible to customer/seller	Gated tab strictly to userRole === admin	FIXED
BUG-05	Handoff	Desktop users unable to phone	Integrated qrcode library & ?product= deep-link	FIXED

All recorded bugs were verified resolved in the final build with zero regressions.

8.1 Technical Challenges Encountered

Engineering an e-commerce platform with embedded security and AI capabilities surfaced three main technical hurdles:

Challenge 1: Firestore & Local State Inconsistencies

When hydrating products from Cloud Firestore, stale local database records occasionally overwrote newly generated studio photography paths, causing broken room photos to render.

Challenge 2: Preventing Security Privilege Leakage

Early iterations of the security intercept modal included an Inspect in SOC button. Non-admin customers could see this action, violating role boundary isolation.

Challenge 3: Frictionless Desktop-to-Mobile Handoff

Enabling shoppers browsing on desktop computers to complete checkout on mobile UPI apps without complex account pairing protocols.

8.2 Architectural Solutions Implemented

Focused engineering solutions were implemented to resolve each technical challenge:

Solution 1: Canonical Asset Reconciliation

Created a resolveGenuineProductImage helper in catalog.ts that normalizes hydrated Firestore documents against verified studio photography paths, preventing stale overrides.

Solution 2: Strict isAdmin Prop Gating

Updated SecurityCheckoutModal.tsx and CartCheckoutDrawer.tsx to receive an explicit isAdmin={userProfile?.role === "admin"} prop, omitting administrative links for regular users.

Solution 3: Vector SVG QR Code Deep-Linking

Integrated the qrcode library to generate vector SVG codes encoding window.location URLs. Added a URL listener in App.tsx to auto-launch the matching product modal upon mobile scan.

8.3 Final System Visual Showcase

The completed PrimeKart web platform demonstrates a harmonious integration of commercial usability and cybersecurity defense.

[SCREENSHOT PLACEHOLDER: Comprehensive Application Showcase]

Capture and paste actual website screenshot here before final submission

Figure 8.1: Full Multi-Panel Application Showcase

Composite showing: 1. Main Storefront Hero, 2. Product Detail Modal with Mobile QR, 3. Cart Checkout Drawer, 4. Security Operations Center, and 5. AI Review Synthesizer.

The showcase highlights how zero-trust security and generative AI can be integrated without disrupting clean consumer aesthetics.

CHAPTER 9: RESULTS & CONCLUSION

9.1 Project Outcomes & Achievements

The internship project successfully satisfied all original technical, security, and user experience objectives:

- Complete Commercial Storefront: Operational React 19 application supporting 4 luxury categories, multi-currency conversion, and persistent shopping carts.
- Cross-Device Mobile QR Handoff: Seamless on-demand SVG QR code generator with deep-linked URL synchronization (?product=<id>).
- Explainable Cybersecurity Engine: Server-authoritative 0-100 risk scoring evaluating bot velocity, payment retries, and device sharing.
- Zero-Trust Role Isolation: Security Operations Center and attack simulation tools strictly masked from Customer and Seller accounts.
- Practical Generative AI: Integration of Google Gemini 3.8 Flash via @google/genai for customer review synthesis and visual search.
- Production Code Quality: Clean static type-checking (tsc --noEmit) and verified bundle compilation.

The platform serves as an end-to-end prototype for secure modern digital retail.

9.2 Learning Outcomes & Future Work

Student Learning Outcomes:

- Advanced React 19 Engineering: Mastered state management, performance optimization, and modular component hierarchy.
- Zero-Trust Web Security: Gained practical expertise in designing server-side risk heuristics and Firestore security rules.
- Cloud Database Management: Learned real-time document synchronization and role-based data partitioning in Cloud Firestore.
- AI SDK Integration: Implemented generative AI using official @google/genai TypeScript SDKs and structured JSON schemas.

Planned Future Work (Future Enhancements):

- Production Payment Gateways: Integrating production Razorpay and Stripe webhook handlers for live financial clearing.
- Hardware FIDO2 Biometrics: Upgrading simulated parent biometric assertions to hardware WebAuthn fingerprint/FaceID sensors.
- Edge CDN Asset Delivery: Caching catalog images across global Content Delivery Networks for sub-50ms international latency.

9.3 Conclusion & Technical References

The development of PrimeKart (by Aetheria) during this internship provided invaluable hands-on experience in full-stack engineering, cloud architecture, and web application security. By combining an intuitive digital storefront with an explainable cybersecurity engine and cloud-backed identity management, the project successfully demonstrated how retail platforms can protect merchants and consumers while delivering an exceptional shopping experience.

Technical References:

- React Documentation: React 19 Component Architecture, Hooks & State Management. (https://react.dev)
- Google GenAI SDK: @google/genai TypeScript Reference and Gemini 3.8 Flash Model Guide. (https://ai.google.dev/gemini-api/docs)
- Google Firebase Documentation: Authentication and Cloud Firestore Security Rules Configuration. (https://firebase.google.com/docs/firestore)
- Tailwind CSS Guidelines: Modern Utility-First Typography and Fluid Layout Systems. (https://tailwindcss.com/docs)
- OWASP Foundation: Automated Threat Handbook for Web Applications & E-Commerce Security. (https://owasp.org)
- Express.js Documentation: Node.js Web Application Framework & Routing Middlewares. (https://expressjs.com)

All technologies and frameworks cited in this report were verified directly from project source code.